Diaspore Team Guide

CS 6650 final project

The Diaspore team

Version 0.3, November 9, 2026

How to read this guide

This guide gets the whole team on the same page. Read it top to bottom once. After that, use it as a map.

Lines marked [**FILL IN: ...**] are open items. We fill them in together, and anyone can edit this guide.

Contents

1 What Diaspore is **2**
2 Five words you will see everywhere **2**
3 How it works **2**
4 The determinism contract **3**
5 Scope **4**
6 The plan **5**
7 Repo and how we work **6**
8 Reading list **7**
9 Who does what **8**
10 Open choices **10**
11 Course deliverables and dates **10**
12 How we talk to each other **11**
13 Glossary **11**

1 What Diaspore is

Diaspore is a tool that makes distributed bugs reproducible.

A distributed bug often depends on timing and message order. That makes it hard to repeat. If you pause a node to debug it, the timing changes, and the bug can hide.

Diaspore fixes this. Every source of randomness in a run comes from one seed, which is one number. A failing run can then replay exactly, on any machine.

2 Five words you will see everywhere

Every name comes from the dandelion. Nouns are botanical. Commands are plain verbs.

Term	What it means in Diaspore
Diaspore	The project and the binary
Floret	One node. A program that speaks the Diaspore protocol
Capitulum	The set of florets in one run
Pappus	The replay file, with the `.pappus` extension
Dandelion	The seed sweep, on one machine or across many

The plural of floret is florets. The plural of capitulum is capitula. Never "capitulums".

Before anyone adds a sixth term, check the rule in `docs/naming.md`. It also lists names we rejected on purpose.

3 How it works

Read `docs/design.md` for the full version. This is the short version.

One binary owns everything. Diaspore is a single Go program. It owns time, the network, faults, client load, and randomness.

Florets are separate programs. They can be written in any language. They talk to Diaspore with JSON lines on stdin and stdout.

Events and replies. A floret only reacts. Diaspore sends it one event, and it answers with one reply. There are three kinds of event: `init`, `message`, and `timer`. The reply is called `done`.

Lockstep. Lockstep means Diaspore sends one event to one floret, then waits for the reply before anything else happens. Only one floret runs at a time. This is what keeps two runs of the same seed identical. The cost is speed, because every event is a round trip to another process. We measure that cost in the report.

Virtual time. Time is a number that Diaspore owns. It jumps straight to the next event. Florets never read a real clock.

Faults. The course version supports five: delay, drop, duplicate, crash, and partition. Rules live in `diaspore.yaml`.

Checkers. A checker reads what clients asked and got back. It decides whether a rule was broken. There are two: `acked-writes` and `linearizability`.

Dandelion across machines (CP24). A coordinator hands out seed ranges to workers on other machines. If a worker dies, its seeds go to another worker. Re-running a seed is safe, because the same seed always gives the same run.

Outputs of a run.

File	What it holds
`trace.jsonl`	Every event in the run. Its hash is the run's fingerprint.
`history.jsonl`	Every client request and result, with times
`*.pappus`	The seed, full config, floret hashes, versions, expected trace hash, and a failure summary

Commands.

```
diaspore run --seed 8837421
diaspore dandelion --seeds 1-1000 --workers 8
diaspore replay run-8837421.pappus
diaspore check-determinism --seed 8837421
```

Why Diaspore counts as a distributed system

The simulator runs one floret at a time on purpose. So one run is not distributed. The sweep is.

With CP24, the coordinator and the workers share no memory and talk only by messages. A worker can die in the middle of a range, and the coordinator must notice and re-send its seeds. That is partial failure. Recovery is simple because of determinism, which is the same idea MapReduce uses when it re-runs a task.

We will not claim internet scale. A few workers can show a trend and a bottleneck. The charts must show where it stops scaling.

4 The determinism contract

Determinism literally means the same inputs always give the same outputs. The whole project depends on it.

Florets must:

1. Do work only inside the handler for the current event.
2. Never read the real clock.
3. Never open network connections.

4. Never use unseeded randomness.
5. Keep anything that must survive a crash in `store`.

Diaspore's core must:

- Never read the real clock.
- Never use global randomness.
- Never run goroutines inside the event loop. Goroutines are allowed only in the process driver, and in the dandelion: between separate seeds, and in the coordinator and workers.

`check-determinism` runs one seed twice and compares the traces byte for byte. CI runs it on every commit.

If you write code in the core, treat these rules as hard limits.

5 Scope

Must-haves. Every must-have ships. The gates in the plan only ever drop bonus items. The one exception is CP24, which has a smaller fallback (Gate 2).

- Core: event loop, virtual time, one seed, and a trace hash.
- Protocol v1, with a spec and JSON Schema files.
- A lockstep process driver with a watchdog.
- Timers, storage, and crash with restart.
- Faults: delay, drop, duplicate, crash, and partition.
- Workload and history, plus two checkers.
- A Go key-value example floret with one planted bug.
- One tiny floret in another language.
- CLI: `run`, `dandelion`, `replay`, and `check-determinism`.
- The `.pappus` v1 format.
- `dandelion` across machines (CP24, formerly bonus B1).
- Scaling measurements for the report, on one machine and across machines.

Bonus, in this order, only if Milestone D lands early:

B2. An HTML timeline viewer.
B3. A GitHub Action that attaches the `.pappus` file.
B4. An optional Go SDK helper.
B5. etcd's Raft library, wrapped as a floret.

Not now: a live production runtime, disk faults, clock skew, shrinking failing runs, and binary encodings.

Limits, stated honestly:

- Diaspore does not test existing services unchanged. Florets must follow the protocol.

- It does not prove a system correct. It finds bugs. It cannot show there are none.
- It does not model real TCP behavior, kernel scheduling, or disk timing.

6 The plan

Work started on Thursday, October 29, 2026, and ends on December 14. The team is three people. The checkpoint details in `docs/build-guide.pdf` (Goal, Why, Decide first, Tests first, Done when) are the source of truth. Replace the copy in the repo with the new version. `docs/plan.md` still shows the old dates. Use the calendar below until it is updated.

Starting point: CP0 is done, and CP1 is next. [**FILL IN: confirm the last finished checkpoint**]

Work left: CP1 to CP24. At 2 to 4 hours each, that is about 48 to 96 hours for the team, or 16 to 32 hours for each of the three of us. CP24 is bigger than most, so plan the high end for it. Hours per person per week: [**FILL IN: hours**]

Project deadlines

Deadlines use Vancouver time. The Vancouver clock stays at UTC-7 all year. After November 1, Boston is two hours ahead, so 9am in Vancouver is 11am in Boston.

Due (Vancouver)	What	Notes
Mon Nov 23, 9am	Project Liftoff: Project Proposal and Team Charter	11am Boston. Graded on on-time delivery and how actionable the plan is.
Mon Dec 14, 9am	Final Project deliverables (20% of the grade)	11am Boston
Mon Dec 14, 1–3pm	Final Project presentations	3–5pm Boston

Draft the Project Proposal and Team Charter by Wednesday, November 18, and submit by Sunday, November 22.

Calendar

Week	Dates	Phase	Checkpoints	Milestone
1	Oct 29–Nov 1	Core	CP1, CP2, CP3, CP7, in parallel (4 days)	
2	Nov 2–8	Core, then protocol	CP4–CP6, CP8, CP9	A: Nov 8
3	Nov 9–15	Protocol	CP10–CP14	B: Nov 15, Gate 1
4	Nov 16–22	First bug	CP15–CP17, CP22, and Project Liftoff	Liftoff due Nov 23
5	Nov 23–29	First bug (light week)	CP18, CP19, CP21, and the CP24 design	C: Nov 29
6	Nov 30–Dec 6	Finish and scale out	CP20, build CP24, then CP23	D: Dec 6, Gate 2
7	Dec 7–13	Wrap-up	Report, poster, demo, buffer	Due Dec 14

Milestones

- **A (Sun Nov 8): deterministic core.** Ping-pong runs over the simulated network. The same seed gives the same hash, every time. The core has no goroutines and no real clock. `PROTOCOL.md` is written.
- **B (Sun Nov 15): real processes in lockstep.** Real floret processes run in lockstep, with timers. Crash and restart work, with storage. `check-determinism` catches a bad floret. Blog post 1 is published.
- **C (Sun Nov 29): the first bug, end to end.** A sweep finds the planted bug. The `.pappus` file replays it with the same hash. A short demo is recorded.
- **D (Sun Dec 6): every must-have is done.** Every must-have works and is tested. `dandelion` runs on at least two machines and four workers, with scaling charts and the bottleneck named. v0.1.0 is tagged, with binaries for Linux, macOS, and Windows.

Gate 1 (Nov 15). If Milestone B is not working, drop bonus items B2 to B5. Put that time into the must-haves. CP24 stays.

Gate 2 (Dec 6). If Milestone D is not done, shrink CP24 but keep its core: the coordinator re-sends a dead worker's seeds to another worker. Drop workers joining in the middle of a sweep, and tuning the range sizes. Dead-worker recovery is the last thing to cut. If it is cut, we write it up as a limit. Every other must-have ships in full.

Why this can work. CP1, CP2, CP3, and CP7 do not depend on each other, so all three of us can start at the same time, with one person taking two of them. CP24 only needs single runs and the `.pappus` file, so its design can start in week 5.

7 Repo and how we work

Repo: `https://github.com/ru-dr/diaspore`

License: MIT.

Get started

```
git clone https://github.com/ru-dr/diaspore.git
cd diaspore
go test ./...
```

If you cannot open the repo, tell the team and we will add you as a collaborator.

Layout, in short

Folder	What it is for
`cmd/diaspore/`	The command line entry points
`internal/`	The private engine. Other projects cannot import it.
`testdata/florets/`	Small test florets. Go tooling skips this folder when building.
`examples/`	The example key-value floret, and the second-language floret
`docs/`	Design, plan, naming, findings, and the build guide
`schema/`	JSON Schema files for the protocol
`runs/`	Run outputs. Listed in `.gitignore`.

The full file-by-file layout is section 15 of `docs/design.md`. Each file shows which checkpoint creates it.

Working rules

- GitHub Flow. `main` is protected. Work on a branch and merge through a pull request. Branch names use a type prefix.
- Draft pull requests are welcome. They start CI early.
- Conventional Commits for commit messages.
- CI runs the tests on Linux, macOS, and Windows on every push. Keep it green.
- Tests first. Every `.go` file gets a matching `_test.go` file next to it.
- One owner for each checkpoint. Someone else reviews the pull request.
- Decisions get written down. Each checkpoint has "Decide first" questions. Record the answer and the reason in the pull request description. A bigger decision becomes an ADR in `docs/adr/`.
- Decisions are shared. Anyone can propose a change to the plan or the design with a pull request or at the weekly sync.
- Seeds that break a checker go in `docs/findings.md`.
- Keep your own work visible in commits, pull requests, and issues. Each person writes a solo reflection, and it needs evidence.
- Naming. Use the vocabulary in Section 2.

8 Reading list

In the repo, in this order

1. `README.md`
2. `docs/naming.md`
3. `docs/design.md`
4. `docs/plan.md`
5. `docs/build-guide.pdf`
6. `PROTOCOL.md` (the message format between Diaspore and florets, written in CP7)

Related work. These are the projects we compare against in the report.

- FoundationDB and TigerBeetle's VOPR: whole-system simulation driven by one seed.
- Jepsen's Maelstrom: nodes as separate programs speaking JSON over stdin and stdout.
- gosim and detsim for Go. turmoil and madsim for Rust.
- Antithesis: a commercial platform for deterministic testing of any software.
- Porcupine (`github.com/anishathalye/porcupine`): the Go library we use for the linearizability checker.

Before we publish any claim about these tools, we check what they support today. In particular: does Maelstrom support exact replay from a seed, and what are the current features of gosim and detsim?

Course background (recommended in the syllabus, not required)

- *Distributed Systems for Fun and Profit*, Mikito Takada (online)
- *Designing Data-Intensive Applications*, Martin Kleppmann

Team research notes and links: [**FILL IN: paste or link the research here**]

9 Who does what

This is a starting proposal. We agree on owners together at the first sync, and anyone can ask to swap. The table follows the calendar in Section 6, and shows what each area needs first.

Area	Checkpoints	Week	Needs first	Owner
Clock, queue, and seed	CP1, CP2, CP3	1	Nothing. These three are independent.	**[FILL IN]**
Protocol spec and JSON Schema	CP7	1	Appendix A of the build guide	**[FILL IN]**
Loop, trace, simulated network	CP4, CP5, CP6	2	CP1 to CP3	**[FILL IN]**
Process driver and lockstep	CP8, CP9	2	Protocol (CP7)	**[FILL IN]**
Timers, storage, crash, and restart	CP10, CP11	3	Lockstep driver (CP9)	**[FILL IN]**
`check-determinism`	CP12	3	Lockstep driver (CP9)	**[FILL IN]**
Config file and faults	CP13, CP14	3	Core, network	**[FILL IN]**
Crash schedule and partitions	CP15	4	Config (CP13)	**[FILL IN]**
Workload and history recording	CP16	4	Network and config	**[FILL IN]**
Example key-value floret with the planted bug	CP17	4	Protocol (CP7)	**[FILL IN]**
Second-language floret	CP22	4	Protocol (CP7), lockstep driver (CP9)	**[FILL IN]**
Acked-writes checker, `.pappus`, replay, first sweep	CP18, CP19	5	History format (CP16)	**[FILL IN]**
Linearizability checker	CP21	5	History format (CP16)	**[FILL IN]**
Dandelion across machines: design	CP24	5	Single runs, `.pappus` contents	**[FILL IN]**
Parallel dandelion and scaling numbers	CP20	6	Working sweep (CP19)	**[FILL IN]**
Dandelion across machines: build, and scaling charts	CP24	6	CP20, and the design	**[FILL IN]**
Docs and release v0.1.0	CP23	6	Everything above	**[FILL IN]**
Timeline viewer, Action, SDK, Raft (bonus)	B2–B5	None	Milestone D done early	**[FILL IN]**
Report, demo, poster, blog posts	Wrap-up	7	Results from the areas above	**[FILL IN]**

A starting split for three people. Person 1 takes the core and network: CP1, CP2, CP4, CP5, CP6, CP14, and CP15, then the design and build of CP24. Person 2 takes the protocol and the process driver: CP7 to CP12, then CP21, CP22, and CP23. Person 3 takes the seed, config, workload, example floret, checker, and sweep: CP3, CP13, CP16, CP17, CP18, and CP19, then CP20 and the scaling numbers. In week 1, Person 1 takes CP1 and CP2, Person 2 takes CP7, and Person 3 takes CP3.

CP24 has the most risk, so Persons 2 and 3 should help Person 1 with it in week 6. Two machines and four worker processes meet the CP24 minimum, so three people can cover it.

Two traps for the second-language floret. Python randomizes string hashing in each process. JavaScript's `Math.random` cannot be seeded. Diaspore has to set up each floret's

environment to avoid these.

10 Open choices

These are not decided yet. They must be locked before the build reaches them.

Choice	Options	Needed by
Language of the second floret	Python or JavaScript	CP22, Week 4 (Nov 16)
How workers talk to the coordinator, and where workers run	`net/http` or `net/rpc`; laptops, cloud instances, or containers	CP24 design, Week 5 (Nov 23)
How people install Diaspore	`go install`, release binaries, or both	CP23, Week 6 (Nov 30)

Two smaller questions are left to checkpoints: whether reordering needs its own fault (CP6, Week 2), and what happens to messages and timers aimed at a crashed floret (CP11, Week 3).

11 Course deliverables and dates

The syllabus is tentative and may change. Check Canvas and Piazza for updates.

- The final project is worth 20% of the course grade.
- Project Liftoff (Project Proposal and Team Charter): Monday, November 23, 9am Vancouver.
- Final project deliverables: Monday, December 14, 9am Vancouver.
- Final project presentations: Monday, December 14, 1 to 3pm Vancouver.
- Thanksgiving is Thursday, November 26. That week is planned light.

Final deliverables

- A link to the shared repository. The README says what Diaspore is and why we built it. The commits, issues, and pull requests show how the project moved over time.
- A project management write-up: how the design moved from the first plan to the final state, who worked on what, and the problems on the way.
- An experiments write-up as a PDF, at most five pages. For each experiment: the purpose, the tradeoff, and the limits. Then the results with charts, and the analysis. The experiment code is in the repository.
- A solo reflection from each person, at most five pages. It is not a team effort.
- The syllabus also lists a poster, a blog report, and the presentation. Details come later in the semester.

Final report outline

1. The problem: why distributed bugs do not replay.
2. The design: virtual time, one seed, lockstep, and the protocol.

3. Why Diaspore is a distributed system: the coordinator and workers, partial failure, and safe re-running of seeds.
4. Faults and checkers.
5. Results: the bug found, the scaling charts for one machine and for several, the bottleneck, and the cost of the process round trip.
6. Limits, stated honestly, plus future work.
7. Related work.

Demo script: sweep, failure, `.pappus` file, replay, and the timeline if B2 exists.

Wrap-up plan: freeze the code and the experiment runs by Wednesday, December 9. Write after the freeze. Keep Saturday and Sunday as buffer, and submit by Sunday night.

12 How we talk to each other

We agree on these together at the first sync. [**FILL IN: chat channel, weekly sync time, how to ask for review on a pull request**]

13 Glossary

Term	**Plain meaning**
Seed	One number. Every random choice in a run comes from it.
Determinism	The same inputs always give the same outputs.
Lockstep	One event goes to one floret, and Diaspore waits for the reply before anything else happens.
Virtual time	A clock that Diaspore moves forward by hand. It never reads the real clock.
Trace	The record of every event in a run. Its hash is the run's fingerprint.
History	The record of every client request and reply, with times.
Checker	A program that reads a history and says whether a rule was broken.
Linearizability	Every operation looks like it happened at one instant between its start and end, as if there were only one copy of the data.
CI	Continuous integration. A server that runs your checks on every push.
Unknown outcome	A request that timed out. The write may or may not have happened. Checkers must treat it that way.
Coordinator	The one process that hands out seed ranges to workers and collects the failures.
Worker	A process, on any machine, that runs the seeds the coordinator gives it.
Bottleneck	The part of the system that limits how fast the whole thing can go.